

Configuration-Based Smart Customization Service: A Multitask Learning Approach

Yue Wang^{ID}, Xiang Li, and Fugee Tsung^{ID}

Abstract—Smart customization service is an important element for smart manufacturing. The success of smart customization requires that designers, manufacturers, and customers with differences in context, semantics, and other cognitive aspects be engaged in a collaborative process. With product configurators reported to have positive impacts on product quality to meet customers' needs, this article attempts to explore an approach for smart customization service based on configurators. To better address the semantic gap between customers and designers/manufacturers, a new configuration mechanism is proposed that takes into consideration customer needs using natural language as the input and maps them to product specifications in the design stage. We collected a massive amount of review text from e-commerce websites and used ELMo, a contextualized word representation based on a deep bidirectional language model, to encode the text. A multitask learning-based neural network was adopted to build the mapping from layman customer needs to product specifications. Our experiments show that this approach can achieve a promising performance for the configuration task and, thereby, facilitate smart customization services.

Note to Practitioners—Smart customization has been adopted by various industries to tailor companies' business streams and customize solutions for customers. It is a complicated service process involving cross-functional teams for identifying customer needs and establishing product design and manufacturing specifications. However, communication in the collaborative process may be challenging. Customers may express their needs using layman's terms. The expressed needs can even be ambiguous and imprecise. Miscommunication hinders the efficiency of smart customization services. This article uses natural language processing and machine learning techniques to map product review text, which is crawled from e-commerce websites to the corresponding product specifications. Given a new customer's needs in free-form text, the mapping can automatically identify the satisfactory product configurations. This has the potential to improve the efficiency of product customization and shield customers and companies from the back-and-forth communication procedure in the customization service.

Manuscript received October 14, 2019; revised February 21, 2020; accepted March 30, 2020. This article was recommended for publication by Associate Editor J. Arinez and Editor Y. Ding upon evaluation of the reviewers' comments. This work was supported in part by the Hong Kong Research Grant Council through the FDS Project (UGC/FDS14/E07/17) under Grant GRF 16201718 and Grant 16203917, and in part by the NSFC under Grant 71931006. (Yue Wang and Xiang Li contributed equally to this work.) (Corresponding author: Yue Wang.)

Yue Wang and Xiang Li are with the Department of Supply Chain and Information Management, The Hong Kong University of Science and Technology, Hong Kong (e-mail: yuewang@hku.hk; lix@hku.hk).

Fugee Tsung is with the Department of Industrial Engineering and Decision Analytics, The Hong Kong University of Science and Technology, Hong Kong (e-mail: season@ust.hk).

Color versions of one or more of the figures in this article are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TASE.2020.2986774

Index Terms—Deep learning, product configurator, smart manufacturing, smart service, transfer learning.

I. INTRODUCTION

SMART service has been a critical element of smart manufacturing. Companies not only need to adopt an advanced manufacturing mechanism, such as cyber-physical systems (CPSs), but must also become more service-oriented [1], [2]. During HANNOVER MESSE 2019, Siemens advocated that, for enterprises in the consumer sector, it is extremely important to conduct large-scale customized product design and manufacturing, improve market response capabilities, and achieve rapid product-innovation iterations [3]. Companies in various industries have made efforts to offer smart service in order to increase the added value for their customers [4].

According to [5], "Smart services are platform-centered value-creation systems based on intelligent products and data-driven services, which offer individual solutions in order to address specific customer needs." To address individual customers' needs in smart service processes, customers must be well integrated into the product life cycle, especially in the initial design stage. Thus, codesign based on a smart-customization platform could be an enabling toolkit for smart service. It is critical to balance customer value, market dynamics, and product technology in one collaborative platform by facilitating product innovation among stakeholders in the whole design cycle by aligning customers' needs with design and manufacturing specifications [9]. However, this is usually a challenging task for manufacturing industries. According to [6], information about customers' needs and companies' solutions is sticky in the sense that they are not in the same semantic space and are difficult to transfer and use in a synthesized way. This semantic gap has been considered a key challenge for smart manufacturing [43]. Thus, there is a need to develop new methods to mitigate sticky information and facilitate smart service and customization.

Current information technology and data analytics techniques could provide a suitable platform for mitigating the sticky-information effect in smart service by synthesizing designers, suppliers, manufacturers, and customers with differences in context, semantics, and other cognitive aspects to a unified framework and gaining consensus during the whole life cycle of the product. However, an effective and smart medium for integrating customers and other stakeholders in the smart-customization service process is currently lacking,

thus often yielding mixed results in smart customization and service processes.

Among existing IT-enabled design toolkits for smart customization, the underlined structure in the product configuration may, potentially, lend itself to facilitating interactions between designers/manufacturers and customers. A product configurator enables matches between each customer's specific needs and the product solution delivered by the company to meet those needs [7]. It consists of a set of well-defined product attributes and their alternatives. Configurators take customers' specifications for attributes as input. The combination of the specifications forms the customized product. In this way, the product-customization process can be considered a set of attribute selection tasks. Although the degree of customization is sacrificed as the solution space is fixed, greater efficiency and benefits can be achieved from a production and manufacturing perspective. Configurators have been acknowledged to shorten product development time, increase customer satisfaction, and reduce the product design cost. In an empirical study on the relationship between product configurators and quality by [8], the use of a product configurator was demonstrated to have positive effects on product quality. Configuration systems have been widely and effectively used in the automobile, machinery, aerospace, and consumer electronics industries, among many others.

However, existing configurators need further improvement in order to serve as the middle ground between customers and other stakeholders in the product-customization process. Essentially, potential challenges and conflicts related to smart customization can be traced to information asymmetry and preferential conflicts between customers, designers, and manufacturers [5]. A key issue in smart customization is how to resolve conflicts since participants' preferences are frequently not fully aligned and uncertainty is involved. Moreover, customers' needs are usually not well-defined [10]. Consequently, achieving clarity for requirements often becomes challenging. Specifically, customers' needs may be expressed in formats that differ from designers' and manufacturers' terminology; commonly, they are expressed in free-form natural language. In addition, the expressed needs may be contradictory and imprecise; this can lead to a semantic gap between the needs expressed by customers and companies' product specifications [11]. To summarize, configurators need to be improved in terms of customers' involvement formats, the determination of the solution space, and the freedom to explore the solution space in order to fully address sticky information issues in smart customization.

With this article, we are prepared to facilitate smart customization services by applying product configuration procedures. Specifically, our aim is to improve users' experiences of configuration processes by allowing them to freely express their needs instead of following the traditional product-specification format. The rationale behind this aim is that customers may not be familiar with the desired product. It is unrealistic to relate their needs to the product attributes when they configure products in the traditional way. We hope to overcome this hurdle with a new configuration mechanism designed to map customers' needs expressed in their natural

language to product attribute specifications. We will leverage the massive numbers of online product reviews and amounts of product metadata to train a deep-learning-based classifier for the mapping. The new configurator will be able to address existing challenges by efficiently bridging the semantic gaps in smart customization between product design specifications and the intentions of customers.

With this article, we have made the following contributions. First, we defined a new configuration mechanism to facilitate smart customization service for product development. This new mechanism is more user-friendly and can, potentially, transform customers' needs that are stated in layman's terms to product specifications. Second, we collected sufficient numbers of product reviews and amounts of product metadata to build the mapping and develop this new configurator. To the best of our knowledge, this is the first data set of its type. Third, we used the powerful pretrained ELMo language model and multitask learning, a major transfer-learning technique, to understand customers' needs in their natural language and build the mapping. The source code and data are released at <https://github.com/yw57721>. We hope that this research can serve as a baseline study for other researchers to follow.

II. RELEVANT RESEARCH

Configurators serve as a critical toolkit that enables customers to cocreate the product. Configurators can facilitate and streamline smart service, such as customized product development, by reducing configuration errors and by enhancing flexibility and responsiveness; thus, they have been widely applied in the industry [7]. In the past three decades, extensive research on product configurator design or configuration has been conducted in the areas of computer science, industrial engineering, and operations management. The research topics include reasoning techniques in configuration, interface design of configurators, and operational issues when implementing configurators, among others. A comprehensive review can be found in [12]. This section will review only works on the use of product configurators for facilitating interactions between customers and other stakeholders in the customization service from the following two perspectives.

A. Improved Efficiency by Defining and Communicating the Solution Space

Without some clarity regarding the solution space, the traditional customization service process left all the parties with a wide-open space to search for the alignment of customers' needs with product design attributes. The process is complicated, costly, and time-consuming, and it requires multiple rounds of product concept validation, feedback, revisions, and modifications [13]. Configurators offer a predefined structure of solution space to assist the consolidation of diverse inputs to determine what will be designed and what will not. A well-defined solution space should include the product attributes or elements along which customers' needs diverge the most in order to meet diversified customer needs [14]. Smart customization can also benefit from well-planned solution spaces. Instead of adopting a design space that mainly takes

into consideration manufacturers' concerns, solution-space development for smart customization can include customers' preferences.

In academia, researchers have also leveraged product configuration data to identify critical attributes for product solution-space development/improvement. Urban and Hauser [15] studied the real-time product configuration data from virtual online product advisers and identified new product-development opportunities in existing solution spaces to address previously unmet customer needs. Wang and Tseng [16] applied the Bayes factor analysis on product configuration data to identify newly emerging customer needs and update the solution space correspondingly. Recent research has begun to deploy user-generated content to identify critical product features and refine the solution space. Archak *et al.* [17] applied the review text to estimate customers' relative preferences to different product features. Wang *et al.* [18] deployed an attention network to map product reviews to product attribute choices. Timoshenko and Hauser [19] found that user-generated content can provide comparable or even better customer-needs information for product development than traditional marketing methodologies. Once the solution space has been determined, it can be used to achieve efficiency, mainly as a result of the better-satisfied customer needs, shortened customization lead time, and costs for future customers.

B. Enhanced User Experience Through Solution-Space Navigation

Communication in traditional service processes involves multiple parties [20]. Each party has its own interests and terminology, which may hinder the communication process due to semantic gaps. In traditional product-configuration processes, the design process is confined to the solution space. Customers need to select the desired attribute choices to develop a satisfactory product. Thus, traditional configurator requires customers to possess a certain amount of expertise and domain knowledge in order to configure a product. Although this requirement is unrealistic in most cases, we argue that user-friendly solution-space navigation has the potential to facilitate smart service. Navigation should allow customers to express their needs in multiple ways instead of the detailed product-design specifications required by configurators. Research in this direction has been conducted to help customers find what they want quickly in a more user-friendly manner in human-computer interface design, product design, and marketing science. For example, Randall *et al.* [21] proposed the needs-based configurator, which comprises a set of descriptions of the product in natural language. Customers need only to indicate the extent to which they agree with the statement, and the satisfactory product will be provided accordingly. Empirical research has shown that the needs-based system achieved higher customer satisfaction among novice users and users with low motivation for processing information [22]. Wang *et al.* [23] also conducted research along these lines. They first extracted keywords mentioned in product reviews and mapped them to product specifications.

Since the expressed needs may be vague and contradictory, a set of recommendations can be proposed to enable customers to resolve conflicts on their own. The recommendations can also leverage customer-preference flexibility and provide a satisfactory design based on incomplete needs expressed. Tihihonen and Felfernig [24] studied the recommendation problem for configurable products. They further investigated machine learning algorithms of nearest neighbor, weighted majority voter, the most popular choice, and naïve Bayes. However, they focused on the recommendation side and used only a simple working example to evaluate recommendation methods. The real-world performance of their algorithms is unknown. Triki *et al.* [25] proposed a similar idea, combining configuration with a recommendation, but did not optimize the configuration process. Their recommendations are made in the traditional way, using knowledge- and content-based approaches, and the cold-start problem persists. Wang and Tseng [26] applied the probability ranking principle in information-retrieval research to the recommendation of configurable products. They proved the optimality of the approach in terms of expected search length. Their results indicated that customers could find the desired product configuration with shortened configuration rounds and, thereby, improve choice-navigation efficiency.

III. MULTITASK LEARNING-BASED CONFIGURATION MECHANISM

A. Problem Definition

The purpose of this article is to transform customers' needs from their natural language to product specifications. Therefore, it is reasonable to apply natural language processing (NLP) techniques to understand utterances in the text describing a customer's needs. Existing research on NLP requires large amounts of data relevant to the research question. However, customers' needs are not publicly available on a large scale. It has been recognized that needs expressed by customers in the free-form text are quite similar to product-review text [19]. Product reviews are obtained easily from e-commerce websites. Thus, this article describes building the mapping from product-review text to product specifications and using this mapping to transform the needs of actual customers into product specifications.

Let R be a product-review corpus. $R = \{R_{ki}\}_{i=1}^{N_k}, k = 1, 2, \dots, n\}$, where R_{ki} is the i th review text for the k th product, and N_k is the number of reviews of the k th product. Each R_{ki} is a sequence of words $(t_1, t_2, \dots, t_{|R_{ki}|})$. A product is defined as a list of predefined attributes $(A_1, A_2, \dots, A_n)$. Each attribute A_i is associated with a set of predefined choices, $\{a_{i1}, a_{i2}, \dots, a_{im_i}\}$, where m_i is the number of choices for attribute A_i . Thus, each product configuration (i.e., product variant) in the product family can be represented by $(a_1, a_2, \dots, a_n)$, where $a_i \in \{a_{i1}, a_{i2}, \dots, a_{im_i}\}$. For example, the product of the PC can be generally represented by the vector (processor, RAM, disk, graphic card, ..., and monitor). One PC configuration may be (i7-8655 processor, 16-GB DDR3 RAM, 256-GB SSD, ..., and 15' monitor).

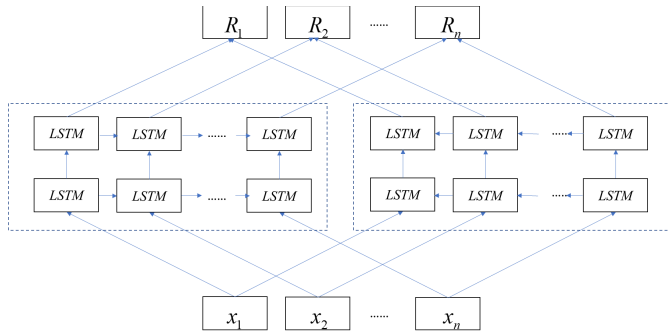


Fig. 1. Structure of ELMo.

For product k , i.e., $(a_{k1}, a_{k2}, \dots, a_{kn})$, it is associated with a set of review text $\{R_{ki}\}_{i=1}^{N_k}$. During the training stage, we attempt to map product reviews to product specifications, $f: R \rightarrow (A_1, A_2, \dots, A_n)$, given the set of product reviews $R = \{\{R_{ki}\}_{i=1}^{N_k}, k = 1, 2, \dots, n\}$ and the corresponding product-configuration set $(A_1, A_2, \dots, A_n)$. This is equivalent to learning a mapping model that assigns a set of labels $\{A_1, A_2, \dots, A_n\}$ to a particular review text. Thus, the configuration task can be modeled as a multilabel text-classification problem. During the testing or application stage, the learned classifier will map a customer's needs text, $Needs_k$, to the most-likely product configuration to be accepted by the classifier $f: Needs \rightarrow (A_1, A_2, \dots, A_n)$.

B. Mapping Reviews to Product Specifications by Multitask Learning-Based Configurators

We plan to deploy multitask learning to implement the mapping $f: R \rightarrow (A_1, A_2, \dots, A_n)$. The network structure of the multitask learning-based approach is shown in Fig. 1. The input to the network is the text of the customer's needs in free form, and the output is the desired product configuration. The basic idea is, first, to obtain an embedding layer of the input text. It should be noted that traditional word embeddings, such as GloVe, are context-independent [27]. Each word is encoded uniquely regardless of the context. For example, the word “window,” which may refer to an operational system or part of a house of other buildings, has a fixed numerical vector to be represented. This reduces the capability of characterizing the meaning in the word. We plan to use ELMo, a context-dependent word representation, to quantify the embeddings of each word [28]. The embedding of the text then goes through a multilayer perceptron (MLP) to extract the characteristics of the text. The MLP output will then be fed into a softmax layer to calculate the probability that each attribute choice satisfies the input needs. Then, the configurations with the largest probabilities are presented.

The details of each layer are as follows.

1) *Embedding Layer Based on ELMo*: Customers' needs are usually presented in free form, which is difficult for computers to process directly. They must be encoded into a suitable format, such as a vector of real numbers so that a computer can easily handle them. The traditional way to accomplish this is to use a one-hot representation. Each word

is represented by a binary vector; only one element in the vector has the value 1, and all the other dimensions have 0 values. The dimension of the vector is equal to the size of the vocabulary. In this way, each word can be encoded uniquely. However, semantic information in the text is missing with a one-hot representation. In addition, the size of the vocabulary may be huge, leading to a high dimension of the one-hot representation. This makes it difficult to operate on the vector.

Recent progress in deep learning has witnessed the popularity of distributed representation of words [29]. Using a two-layer neural network, each word can be transformed into a vector for which the dimension is much lower than for one-hot representation. It is now widely used in natural language processing research, such as GloVe [27]. Using the distributed representation, each word is encoded by a numerical vector. However, these methods cannot manage polysemy very well. For example, the word “apple” can refer to the company or the fruit. However, the traditional word embedding gives “apple” a fixed numerical vector representation. ELMo, which was proposed to solve this problem, is a contextualized word representation based on a deep bidirectional language model [28]. The model is pretrained on a large corpus of text. The syntax and semantics in the text can be incorporated in the model, and the linguistic context is also considered in ELMo. Thus, ELMo is no longer a vector of word embedding but a function of the whole inputted sentence based on a language model. When using ELMo, we need only to input a sentence or a paragraph in ELMo, and it will infer the context-aware embedding of each word. One of the obvious benefits of ELMo is that polysemous words can be understood by considering their context.

The training of ELMo can be illustrated in Fig. 1. Given a sequence of words in a text $(t_1, t_2, \dots, t_n)$, a language model computes the probability that word t_k appears at the k th position given this history $(t_1, t_2, \dots, t_{k-1})$ for a forward language model or the future sequence $(t_{k+1}, t_{k+2}, \dots, t_N)$ for a backward language model. Rigorously, for a forward LM, we have $p(t_1, t_2, \dots, t_N) = \prod_{k=1}^N p(t_k | t_1, t_2, \dots, t_{k-1})$. For a backward LM, $p(t_1, t_2, \dots, t_N) = \prod_{k=1}^N p(t_k | t_{k+1}, t_{k+2}, \dots, t_N)$.

In ELMo, the language models are computed using an L -layer bidirectional long short-term memory (BiLSTM) recurrent neural network (RNN), as shown in Fig. 1.¹ Note that only two BiLSTM layers are shown. The actual network can be deeper. The k th word, i.e., t_k , in the word sequence is encoded via word embedding, x_k , and inputted to the forward and backward LSTMs. Then, the j th layer of the forward and backward LSTM layers output the representations $\vec{h}_{k,j}$ and $\overleftarrow{h}_{k,j}$, which are context-dependent, of the representations of word t_k .

All the L layers and the input layer's outputs are combined to form the ELMo representation of each word in the sequence. The rationale is that, in a deep neural network for language processing, the beginning layers often contain syntax and grammatical information, while later layers contain semantic information. Combining all the layers can comprehensively

¹LSTM is the typical recurrent neural network (RNN) unit. It transforms the raw data into abstract representations for further procedures. The details of LSTM can be found in the appendix.

encode all the information in the text. Rigorously, $R_k = \{x_k, \vec{h}_{k,j}, \overleftarrow{h}_{k,j} | j = 1, 2, \dots, L\} = \{h_{k,j} | j = 1, 2, \dots, L\}$, where $h_{k,j} = [\vec{h}_{k,j}; \overleftarrow{h}_{k,j}]$ for the BiLSTM layer.

It should be noted that there are $2L + 1$ vectors using the L layer BiLSTM deep-network-based language model. They are concatenated into one long vector as the encoding of the words. Since each reviews/needs text contains multiple words, we calculate the simple pointwise average of each word's ELMo vector as the embedding of the whole set of reviews/needs. The review embeddings are then fed into the following multitask neural network to train the mapping model.

2) *Multitask Learning*: Multitask learning is one of the transfer learning techniques, which aims to apply knowledge in one domain to another different but relevant task [37], [38]. Transfer learning can well handle data scarcity issues and has been successfully applied in many industries [39], [40]. In multitask learning, multiple related tasks are learned together based on shared representations of information in the data [30]. Multitasking involves multiple simultaneous tasks and parallel learning. Multiple tasks facilitate each other's learning through the underlying shared representations and, thereby, enhance the generalization effect. Put simply, multitask learning combines multiple related tasks. The learning process enhances the generalization capability by sharing and complementing domain-related information in a shared representation [31].

Considering the configuration task in this article, we aim to map product reviews to product specifications, $f : R \rightarrow (A_1, A_2, \dots, A_n)$, given the set of product reviews $R = \{\{R_{ki}\}_{i=1}^{N_k}, k = 1, 2, \dots, n\}$. It should be noted that product attributes are highly correlated. For example, a laptop with a high-end processor is very likely to be paired with good RAM and other specifications. Thus, building $f_i : R \rightarrow A_i$ separately and combining them to form $f : R \rightarrow (A_1, A_2, \dots, A_n)$, as shown in Fig. 2(a), ignore the rich information gained from the training data of related tasks. Thus, we would like to train the relevant tasks simultaneously, fully leverage the domain-specific information, and improve the generalization capability of the model, as shown in Fig. 2(b). The trained multitasking model shares the representation and characteristics of relevant tasks. The joint model can be learned and applied to each of the subproblems.

Multitask learning can be formally defined as follows. Suppose that we have a data set S that contains data pairs in the form of $(\mathbf{x}, \mathbf{y})$, where $\mathbf{x}$ is the input vector, $\mathbf{y} \in \mathbf{Y} = (y_1, y_2, \dots, y_N)$ is the target vector, and y_i represents the i th component of $\mathbf{Y}$.

A single-target task contains only one output y_i , which is based on the function learned from the set S , $f_i : \mathbf{x} \rightarrow y_i$. The objective function is to minimize $\sum_{(\mathbf{x}, \mathbf{y}) \in S} L_i(f_i(\mathbf{x}), y_i)$.

However, multitask learning has multiple outputs $(y_1, y_2, \dots, y_N)$ through the set of functions learned from the data set S $f : \mathbf{x} \rightarrow \mathbf{y}$. The objective function is to minimize $\sum_{(\mathbf{x}, \mathbf{y}) \in S} L(f(\mathbf{x}), \mathbf{y})$. For a typical model, multitask learning is generally better than single-task learning as $L(f(\mathbf{x}), \mathbf{y}) < \sum_i L_i(f_i(\mathbf{x}), y_i)$.

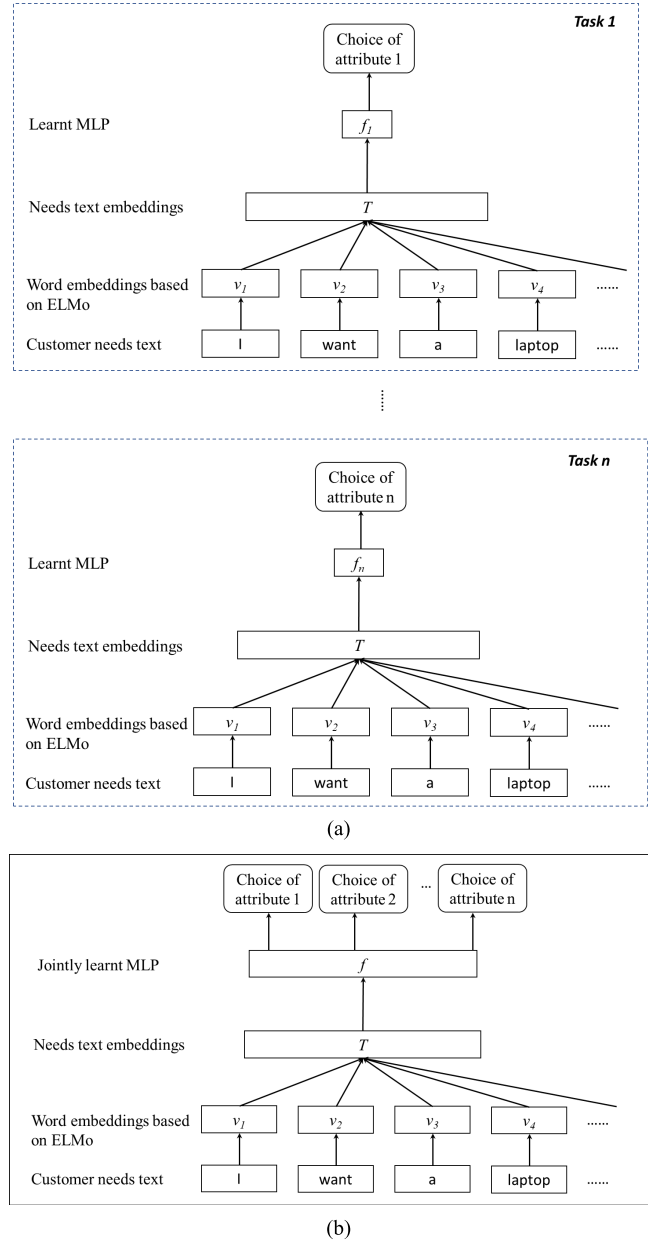


Fig. 2. Structure of (a) single-task learning and (b) multitask learning.

3) *Multiplerlayer Perceptron-Based Multitask Classifier*: The concatenation of the $2L+1$ vectors produced by the $2L+1$ layer BiLSTM is fed into a classifier to realize multitask learning. In this article, we use a simple neural network structure, MLP, to realize multitask learning, i.e., f_i and f are implemented using MLP, as shown in Fig. 2.

MLP is a typical feedforward neural network structure indicated in Fig. 3. It consists of input layers, hidden layers, and output layers. Based on ELMo, each input word t_k in a review text has a contextual dependent representation R_k , as shown in Fig. 1. We use the simple elementwise average of R_k to encode the whole review text. Thus, the review embedding has the same dimension R_k .

Let a review's embedding be represented by $\mathbf{x}_1$ (equivalent to T in the above figure) and inputted into the MLP, where

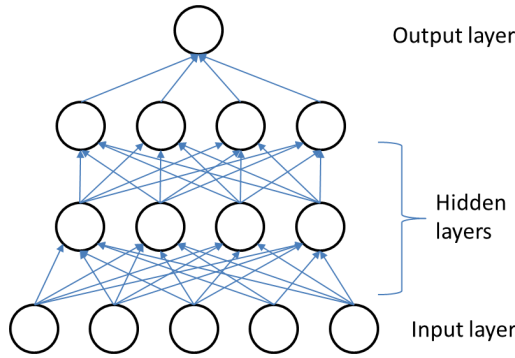


Fig. 3. Structure of MLP.

the index of 1 represents the first layer of the MLP, i.e., the input layer. Then, the hidden layer is calculated as $\mathbf{x}_i = \sigma(\mathbf{W}^i \mathbf{x}_{i-1} + \mathbf{b}^i)$, where $\mathbf{W}^i$ is the matrix connecting the $(i-1)$ th layer with the i th layer. $\mathbf{b}^i$ represents the bias vector of layer i . We use ReLU as the activation function for the hidden layers σ . That is, $\sigma(\mathbf{x}) = \max(\mathbf{0}, \mathbf{x})$. The training of the model, i.e., the identification of all the parameters of $\mathbf{W}^i$ and $\mathbf{b}^i$, is processed by minimizing the cross entropy between the predicted and true label distributions using the backpropagation algorithm [32].

IV. EXPERIMENT SETTING

A. Data Preparation

We crawled the review text for a laptop from amazon.com and used it as training data to build the multitask learning model. Laptops/PCs are well-modularized products. They were among the first batch of products to successfully implement the configure-to-order mechanism. Current laptop/PC manufacturers still utilize this mechanism. Thus, we used it as an example to demonstrate the idea of needs-based configurators for smart customization service. We collected the reviews of 91 laptop models introduced from January 2017 to June 2018. We crawled the product review text from amazon.com. There were 5526 reviews in total.

Each review text is associated with a label vector, i.e., the corresponding product's technical specifications. We considered five major technical specifications for the laptop, i.e., processor, RAM, disk, monitor, and graphics card, as these are customers' main concerns when purchasing a laptop. The number of choices for each component is as follows: ten for processors, six for RAM, eight for disk, six for the monitor, and eight for the graphics card. It should be noted that all the attribute choices are processed so that similar choices are combined. For example, Intel Core i7-8565U and i7-8559U are quite similar in terms of performance and price. Thus, they are considered as one choice. Other attributes, such as wireless type and hard disk type, are seldom mentioned by customers. It should be noted that functional requirements, such as portability and performance, are highly correlated with the five attributes. For example, portability is determined mainly by screen size; performance is determined mainly by the processor, RAM, and so on. Thus, the labels for the review text, namely, the five technical specifications, are

also well-justified. The combination of the technical attribute specifications forms the product configuration.

Since the purpose of the needs-based configurator is to map the needs text to product configurations, we collected a number of authentic customer-needs texts to test the needs-based configurator. An online survey was conducted; each participant was required to input his or her needs for a laptop using layman's language. Then, they were required to identify the most satisfactory laptop from the 91 models.

B. Performance Measure

The output of the needs-based configurator is a set of technical specifications for laptop choices, which are presented based on the probability of relevance calculated by the softmax layer in the neural network. All the possible attribute choices are listed in descending order of likelihood to be the satisfactory choice, i.e., the attribute choice that is likely to be the most satisfactory is at the top of the configuration result list, followed by the second most likely satisfactory attribute choice and so on. This mechanism is similar to an information retrieval system, such as that used by Google. Customers may screen a certain number of choices to find a satisfactory choice. We would like to use Recall@k, the widely used performance metric in information retrieval and classification, as the metric to measure the performance of the configurator, where k is the number of choices that a customer is willing to screen [33].

For each attribute choice i , such as a DDR4 8-GB RAM for the attribute of RAM, the corresponding recall at k is defined as $\text{Recall}_i@k = (TP_{i,k}) / (TP_{i,k} + FN_{i,k})$, where $TP_{i,k}$ is the amount of needs text, i.e., the testing data that are correctly mapped to choice i and choice i is among the top- k items of the configurator's output. $FN_{i,k}$ is the number of testing data that are not mapped to choice i but actually should be mapped to choice i among the top- k output of the configurator. The abovementioned definition can be generalized to multiclass case as $\text{Recall}@k = (\sum_{i=1}^M TP_{i,k}) / (\sum_{i=1}^M (TP_{i,k} + FN_{i,k}))$, where M is the number of classes.

Recall@k quantifies the power of the configurator to identify the satisfactory attribute choices from the top- k items. Besides Recall@k, Precision@k is also widely used to evaluate an information retrieval system. Precision@k is used to quantify the percentage of satisfactory items in the top- k output of the system. In the study of product configuration and information recommendation, it has been acknowledged that users are usually more concerned with whether there is ONE satisfactory item among the top- k ones instead of the number of satisfactory recommendations [34]. In addition, each customer reviews/needs text only corresponds to one product configuration. Thus, Recall@k is more suitable for this problem.

It should be noted that we assume that each user is satisfied with the components contained in his/her selected laptop only. Any other component choices are not acceptable. This is actually a very strict assumption, as customers may be satisfied with more than one attribute choice. Correspondingly, the performance measure is quite strict; therefore, the actual performance should be no worse than the result presented in this article.

TABLE I
EXPERIMENT RESULT

	Processor	RAM	Disk	Monitor	Graphic card
Recall@1	0.333	0.654	0.466	0.736	0.590
Recall@2	0.523	0.818	0.739	0.879	0.853
Recall@3	0.648	0.924	0.838	0.953	0.929
Recall@4	0.739	0.976	0.914	0.978	0.974
Recall@5	0.816	0.996	0.944	0.993	0.991

V. EXPERIMENT RESULTS AND DISCUSSION

A. General Findings

The performance of Recall@k for all the five attributes is shown in Table I. We have achieved very good results for the attributes of RAM, monitor, and graphic card. The first presented choice can satisfy customers with a probability nearly 0.6 (for the graphic card) or above (for RAM and monitor). The top-three presented choices can achieve more than a 90% recall rate. The performance of the disk is not so good but not significantly worse. When $k > 3$, the recall rate is above 90% as well. The performance of the processor is the worst among the five attributes. The reason is that there are more processor choices than other attributes. Originally, there are nearly 30 processors in the 91 laptop models. After data processing, there are ten processor choices (categories). The boundary between these ten choices is not very clear. Thus, the performance of the processor's mapping is not as good as others. However, the performance of the processor is much better than random selection. For example, the Recall@1 for random selection should be $1/10 = 10\%$, which is lower than 33.3% in our experiment. When $k > 1$, the recall rate is already larger than 50% and reach 81.6% when $k = 5$. In summary, the ELMo + multitask framework can achieve very good performance to map customer needs in natural language to product attribute choices.

B. Performance Improvement Resulting From Using ELMo

The highlight of the proposed method is the adoption of ELMo embedding and multitask learning. ELMo takes into consideration the contextual information when encoding each word. Multitask learning has better generalization capabilities by fully leveraging the interdependency among the individual tasks. In this section and the next section, we test the effectiveness of ELMo and multitask learning, respectively.

To demonstrate the superiority of ELMo with traditional context-independent word embedding, we compare the performance of ELMo + multitask learning with GloVe + multitask learning. The gloVe is a popular word embedding developed by Stanford University. It cannot address contextual information, but rather it uniquely encodes each word with a numerical vector.

The performance of Recall@k for these two approaches is shown in Fig. 4. We can see that ELMo + multitask learning outperforms GloVe + multitask learning for all the specifications, although not significantly. When checking some review text, we found that there are polysemous words, such as the words "window" and "apple." ELMo calculates

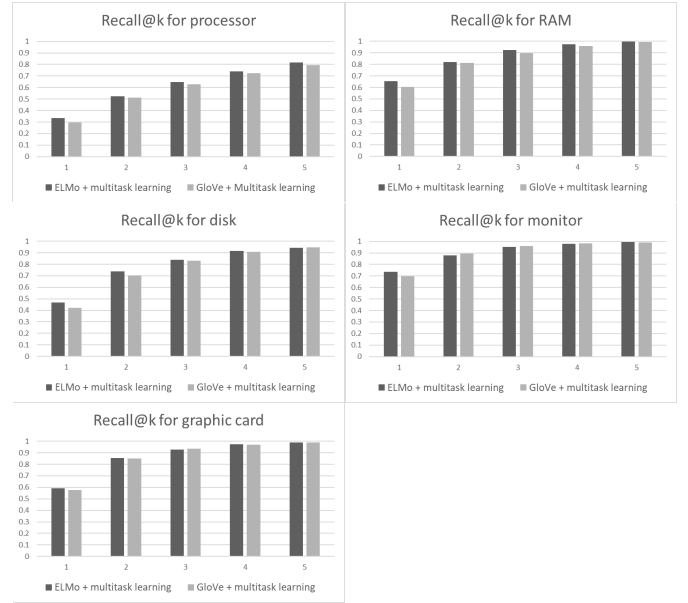


Fig. 4. Comparison of Recall@k between ELMo + multitask learning and GloVe + multitask approaches.

the embedding of each word using the whole sentence or paragraph; it is context-aware. It is obvious that polysemous words can be understood by considering the context. However, we also found that the number of polysemous words is not in a big quantity. This explains why the performance difference between ELMo and GloVe is not significant.

We also notice that when k is small, ELMo's advantage over GloVe is relatively bigger. When k is bigger, the advantage is gradually diminishing. However, for the difficult mapping, such as the mapping for the processor, the advantage of ELMo is stable even when k is big.

C. Performance Improvement Caused by the Structure of Multitask Learning

To show the effectiveness of multitask learning, we compare ELMo + multitask learning with ELMo + single-task learning. In single-task learning, the mapping from reviews/needs to product specifications is treated separately, as shown in Fig. 5. There is no sharing of the information between the neural networks. To make a fair comparison, we use the same network structure for single- and multitask learning, i.e., the two-layer MLP.

Based on Fig. 5, we find that multitask learning dominates single-task learning with big margins for all the comparisons. The reason is that there are relevant and irrelevant elements among multiple tasks in multitask learning. When putting all the tasks in one framework, the relevant information for each product attribute will complement each other to make the mapping more accurate. The irrelevant information among the tasks will be treated as noise during the learning process. The introduced noise could improve the generalization capability of the learned model.

From the model training's perspective, the neural network-based single-task learning tends to make the solution fall into

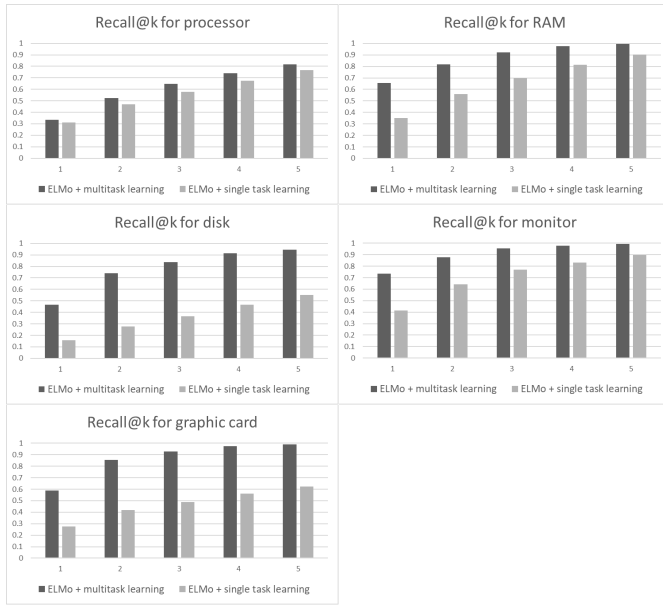


Fig. 5. Comparison of Recall@k between ELMo + multitask learning and ELMo + single task learning approaches.

the local minima due to the property of the backpropagation process when updating the parameters in the model. In multitask learning, the local minimum values of different tasks are located in different positions. This can help the local minimum values to move and converge to the global maxima. In addition, combining multiple relevant tasks can change the dynamics of the weight updating in the learning process. This makes the network more suitable for multitask learning. For example, multitask parallel learning improves the learning rate of the shared representation in the hidden layer. All of these reasons explain why multitask outperform single task learning with a big margin.

VI. CONCLUSION

Smart customization is a complicated service process involving cross-functional teams to identify customer needs and establish product design and manufacturing specifications throughout the entire life cycle of the product [36]. This article aimed to develop new approaches for smart customization that can best meet the requirements of customers and the capabilities of manufacturers simultaneously. Although design and manufacturing have long been recognized as collaborative activities, the progress of developing methods to facilitate smart customization has been hampered by the difficulties related to building common bases among different stakeholders. This article intends to circumvent the abovementioned difficulties by applying and extending the established structure and mechanism of configuration design. Specifically, a novel configuration mechanism is proposed that can transform customer's needs provided in layman's language into product specifications in the early product-design stage instead of selecting one desired option in a traditional configurator. It provides a more user-friendly way of eliciting customers' needs and identifies a satisfactory solution. Instead of selecting

one desired option in a traditional configurator, we leverage the ELMo model and multitask learning to build the mapping based on the review text crawled from e-commerce websites. Online surveys are conducted to collect the testing data. Through the experiment, we found that the method can achieve high efficiency in terms of accuracy and recall rate, as well as the required input from customers. The superiority of the proposed methods is due to two factors: the incorporation of the context-dependent features captured by ELMo, and the generalization capability of multitask learning by considering the dependence among the tasks.

This article still has limitations. The current training data set is a moderate size; this restricts the application of more powerful deep-learning approaches to this smart customization task. In our future work, we hope to collect more high-quality data to build the mapping. We also observed data imbalance issues in our data set. The laptop models in the data set were launched from Jan 2017 to the mid of 2018. The earlier models have more review text than the new models. Thus, the trained model tends to favor old models. We will also explore various methods to handle the data imbalance issue. In addition, although product reviews and customers' needs are similar, they are still different. Another research direction is to use transfer learning to fine-tune the learned mapping using customer needs data to build authentic customer needs to product specification mapping. It is anticipated that this could lead to better performance for the smart customization service for customer-oriented industries.

APPENDIX LSTM BASICS

LSTM is a widely used RNN unit. It transforms the raw data into abstract representations for further procedures [41]. Due to its superior performance, it has been widely used in natural language processing, computer vision, and other areas [42].

An LSTM unit consists of an input gate, an output gate, a forget gate, and a cell. The input to the LSTM unit is x_t , and the outcome is represented by y_t at time t . The information in the data source is stored in the cell at time t . The input gate controls the type of new information to be stored in the LSTM cell. The outputted information is determined by the output gate. The forget gate is used to determine which information from the previous iteration should be discarded. The whole unit works recurrently. The current state output is used as an input to the next iteration. Thus, we have $y_t = \text{LSTM}(y_{t-1}, x_t)$. An LSTM unit at iteration t is shown in Fig. 6. The gates and cells are updated as follows:

$$\begin{aligned}
 z_t &= \sigma(W_z x_t + R_z y_{t-1} + b_z) \\
 i_t &= \sigma(W_i x_t + R_i y_{t-1} + p_i \otimes c_{t-1} + b_i) \\
 f_t &= \sigma(W_f x_t + R_f y_{t-1} + p_f \otimes c_{t-1} + b_f) \\
 c_t &= i_t \otimes z_t + f_t \otimes c_{t-1} \\
 o_t &= \sigma(W_o x_t + R_o y_{t-1} + p_o \otimes c_t + b_o) \\
 y_t &= o_t \otimes h(c_t)
 \end{aligned}$$

where $W_z, W_i, W_f, W_o, R_z, R_i, R_f, R_o$ are the weight matrices for each gate's input and recurrent parts; b_z, b_i, b_f, b_o

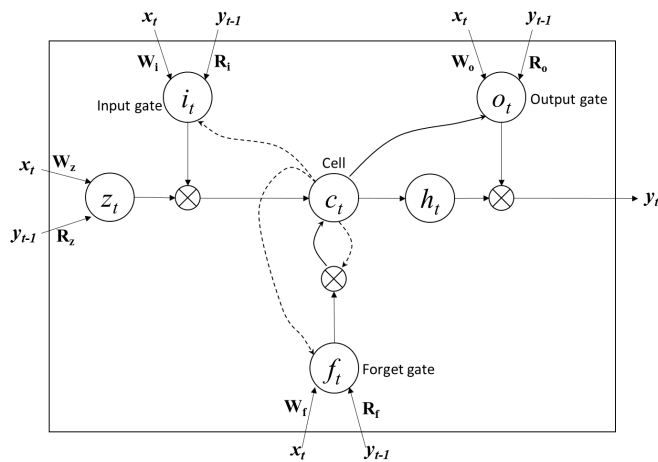


Fig. 6. LSTM unit (the dash lines indicate the time-lag input).

denote the bias vector of each gate; σ and h are the sigmoid and tanh functions, respectively; $\otimes$ represent the elementwise multiplication of two vectors; and p_z, p_i, p_f, p_o denote the peephole connections in the network.

REFERENCES

- [1] Y. Liao, F. Deschamps, E. D. F. R. Loures, and L. F. P. Ramos, "Past, present and future of industry 4.0—A systematic literature review and research agenda proposal," *Int. J. Prod. Res.*, vol. 55, no. 12, pp. 3609–3629, Jun. 2017.
- [2] P. Jussen, J. Kuntz, R. Senderek, and B. Moser, "Smart service engineering," in *Proc. 11th CIRP Conf. Ind. Product-Service Syst. Smart Service Eng.*, 2019, pp. 384–388.
- [3] (2019). *Online Resource: HANOVER MESS*. Accessed: May 30, 2019. [Online]. Available: [https://www.siemens.com/press/en/events/2019/digitalfactory/2019-04-hannovermesse.php?content\[\]=DF&content\[\]=PD&content\[\]=EM](https://www.siemens.com/press/en/events/2019/digitalfactory/2019-04-hannovermesse.php?content[]=DF&content[]=PD&content[]=EM)
- [4] S. Mayer, R. Verborgh, M. Kovatsch, and F. Mattern, "Smart configuration of smart environments," *IEEE Trans. Autom. Sci. Eng.*, vol. 13, no. 3, pp. 1247–1255, Jul. 2016.
- [5] K. Exner, E. Smolka, T. Blüher, and R. Stark, "A method to design smart services based on information categorization of industrial use cases," in *Proc. 11th CIRP Conf. Ind. Product-Service Syst. Smart Service Eng.*, 2019, pp. 77–82.
- [6] E. v. Hippel, *Democratizing Innovation*. Cambridge, MA, USA: MIT Press, 2005.
- [7] M. Sabin and R. Weigal, "Product configuration framework—A survey," *IEEE Intell. Syst.*, vol. 13, no. 4, pp. 42–49, Jul./Aug. 1998.
- [8] A. Trentin, E. Perin, and C. Forza, "Product configurator impact on product quality," *Int. J. Prod. Econ.*, vol. 135, no. 2, pp. 850–859, Feb. 2012.
- [9] R. El Hadj Khalaf, B. Agard, and B. Penz, "Module selection and supply chain optimization for customized product families using redundancy and standardization," *IEEE Trans. Autom. Sci. Eng.*, vol. 8, no. 1, pp. 118–129, Jan. 2011.
- [10] P. Zipkin, "The limits of mass customization," *Sloan Manage. Rev.*, vol. 42, no. 3, pp. 81–87, Apr. 2001.
- [11] T. Hu, J. Zhao, and D. Zhao, "A study on the semantic gap between designer and user in automobile design," in *Proc. Int. Assoc. Societies Design Res.*, Tokyo, Japan, 2013, pp. 1–8.
- [12] L. L. Zhang, "Product configuration: A review of the state-of-the-art and future research," *Int. J. Prod. Res.*, vol. 52, no. 21, pp. 6381–6398, Nov. 2014.
- [13] Y. Wang and M. M. Tseng, "Adaptive attribute selection for configurator design via Shapley value," *Artif. Intell. Eng. Design, Anal. Manuf.*, vol. 25, no. 2, pp. 185–195, May 2011.
- [14] F. Salvador, P. M. De Holan, and F. Piller, "Cracking the code of mass customization," *MIT Sloan Manage. Rev.*, vol. 50, no. 3, pp. 71–78, 2009.
- [15] G. L. Urban and J. R. Hauser, "'Listening In' to find and explore new combinations of customer needs," *J. Marketing*, vol. 68, no. 2, pp. 72–87, Apr. 2004.
- [16] Y. Wang and M. M. Tseng, "Identifying emerging customer requirements in an early design stage by applying Bayes factor-based sequential analysis," *IEEE Trans. Eng. Manag.*, vol. 61, no. 1, pp. 129–137, Feb. 2014.
- [17] N. Archak, A. Ghose, and P. G. Ipeirotis, "Deriving the pricing power of product features by mining consumer reviews," *Manage. Sci.*, vol. 57, no. 8, pp. 1485–1509, Aug. 2011.
- [18] Y. Wang, W. Zhao, and W. X. Wan, "Needs-based product configurator design for mass customization using hierarchical attention network," *IEEE Trans. Autom. Sci. Eng.*, early access, Jan. 6, 2020, doi: 10.1109/TASE.2019.2957136.
- [19] A. Timoshenko and J. R. Hauser, "Identifying customer needs from user-generated content," *Marketing Sci.*, vol. 38, no. 1, pp. 1–20, Jan. 2019.
- [20] J. P. C. Tong, V. G. Duffy, G. W. Cross, F. Tsung, and B. P. C. Yen, "Evaluating the industrial ergonomics of service quality for online recruitment websites," *Int. J. Ind. Ergonom.*, vol. 35, no. 8, pp. 697–711, Aug. 2005.
- [21] T. Randall, C. Terwiesch, and K. T. Ulrich, "User design of customized products," *Market. Sci.*, vol. 26, no. 2, pp. 268–280, 2007.
- [22] M. Weinmann, M. Hibbeln, and S. Robra-Bissantz, "Customer-oriented product configuration systems: One type fits All," in *Proc. 19th Eur. Conf. Inf. Syst.*, 2011. [Online]. Available: <https://aisel.aisnet.org/cgi/viewcontent.cgi?article=1131&context=ecis2011>
- [23] Y. Wang, D. Y. Mo, and M. M. Tseng, "Mapping customer needs to design parameters in the front end of product design by applying deep learning," *CIRP Ann.*, vol. 67, no. 1, pp. 145–148, 2018.
- [24] J. Tiihonen and A. Felfernig, "Towards recommending configurable offerings," *Int. J. Mass Customisation*, vol. 3, no. 4, p. 389, 2010.
- [25] C. Salinesi, R. Triki, and R. Mazo, "Combining configuration and recommendation to define an interactive product line configuration approach," 2012, *arXiv:1206.2520*. [Online]. Available: <http://arxiv.org/abs/1206.2520>
- [26] Y. Wang and M. M. Tseng, "Customized products recommendation based on probabilistic relevance model," *J. Intell. Manuf.*, vol. 24, no. 5, pp. 951–960, Oct. 2013.
- [27] J. Pennington, R. Socher, and C. Manning, "Glove: Global vectors for word representation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1532–1543.
- [28] E. P. Matthew *et al.*, "Deep contextualized word representations," in *Proc. NAACL*, 2018, pp. 2227–2237.
- [29] J. Turian, L. Ratinov, and Y. Bengio, "Word Representations: A simple and general method for Semisupervised learning," in *Proc. 48th Annu. Meeting Assoc. Comput. Linguistics*, 2010, pp. 384–394.
- [30] R. Caruana, "Multitask learning," *Mach. Learn.*, vol. 28, no. 1, pp. 41–75, 1997.
- [31] Y. Zhang and Q. Yang, "A survey on multi-task learning," 2017, *arXiv:1707.08114*. [Online]. Available: <http://arxiv.org/abs/1707.08114>
- [32] V. Nair and G. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. 7th Int. Conf. Mach. Learn.*, Haifa, Israel, Jun. 2010, pp. 807–814.
- [33] D. M. W. Powers, "Evaluation: From precision, recall and F-measure to ROC, informedness, markedness & correlation," *J. Mach. Learn. Technol.*, vol. 2, no. 1, pp. 37–63, 2011.
- [34] H. Chen and D. R. Karger, "Less is more: Probabilistic models for retrieving fewer relevant documents," in *Proc. 29th Annu. Int. ACM SIGIR Conf. Res. Develop. Inf. Retr. SIGIR*, 2006, pp. 429–436.
- [35] R. Collobert and J. Weston, "A unified architecture for natural language processing: Deep neural networks with multitask learning," in *Proc. 25th Int. Conf. Mach. Learn. ICML*, 2008, pp. 160–167.
- [36] C. da Cunha, B. Agard, and A. Kusiak, "Design for cost: Module-based mass customization," *IEEE Trans. Autom. Sci. Eng.*, vol. 4, no. 3, pp. 350–359, Jul. 2007.
- [37] F. Tsung, K. Zhang, L. Cheng, and Z. Song, "Statistical transfer learning: A review and some extensions to statistical process control," *Qual. Eng.*, vol. 30, no. 1, pp. 115–128, Jan. 2018.
- [38] G. Wang, J. Qiao, J. Bi, W. Li, and M. Zhou, "TL-GDBN: Growing deep belief network with transfer learning," *IEEE Trans. Autom. Sci. Eng.*, vol. 16, no. 2, pp. 874–885, Apr. 2019.
- [39] L. Cheng, F. Tsung, and A. Wang, "A statistical transfer learning perspective for modeling shape deviations in additive manufacturing," *IEEE Robot. Autom. Lett.*, vol. 2, no. 4, pp. 1988–1993, Oct. 2017.

- [40] H. Yoon and J. Li, "A novel positive transfer learning approach for telemonitoring of Parkinson's disease," *IEEE Trans. Autom. Sci. Eng.*, vol. 16, no. 1, pp. 180–191, Jan. 2019.
- [41] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [42] K. Greff, R. K. Srivastava, J. Koutnik, B. R. Steunebrink, and J. Schmidhuber, "LSTM: A search space odyssey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 10, pp. 2222–2232, Oct. 2017.
- [43] "Toward smart manufacturing with data and semantics," eClass e.V. Assoc., Cologne, Germany, White Paper, 2018.



Yue Wang received the B.S. degree in electronics and the M.E. degree in information science from Peking University, Beijing, China, in 2002 and 2005, respectively, and the Ph.D. degree in industrial engineering from The Hong Kong University of Science and Technology, Hong Kong, in 2010.

He is currently an Assistant Professor with the Department of Supply Chain and Information Management, The Hang Seng University of Hong Kong, Hong Kong. His research interests include machine learning, natural language processing, design and manufacturing informatics, healthcare informatics, and supply chain management.



Xiang Li received the B.S. degree in Internet-of-Things engineering from Central South University, Changsha, China, in 2018, and the M.Sc. degree in information technology from The Hong Kong University of Science and Technology, Hong Kong, in 2019.

He is currently a Research Assistant with the Department of Supply Chain and Information Management, The Hang Seng University of Hong Kong, Hong Kong. His research interests focus on data mining and natural language processing.



Fugee Tsung received the B.Sc. degree from National Taiwan University, Taipei, Taiwan, in 1990, and the M.Sc. and Ph.D. degrees from the University of Michigan, Ann Arbor, MI, USA, in 1993 and 1997, respectively.

He is currently a Chair Professor and the Acting Dean of the Information Hub, Guangzhou Campus, The Hong Kong University of Science and Technology (HKUST), Hong Kong. He is also the Director of the Quality and Data Analytics Lab. He has authored over 100 refereed journal publications. His research interests include industrial big data and quality analytics.

Dr. Tsung has been elected as an Academician of the International Academy for Quality (IAQ), a fellow of the American Statistical Association (ASA), the Institute of Industrial and Systems Engineers (IISE), the American Society for Quality (ASQ), and the Hong Kong Institution of Engineers (HKIE), and an elected member of the International Statistical Institute (ISI). He received the IISE Transactions' Best Paper Award, three times, in 2004, 2009, and 2018, respectively. He is also the former Editor-in-Chief of the *Journal of Quality Technology* (JQT).